

Chapter 4: Database Abstraction

Gregory S. DeLozier, PhD

The pet application already lets us create, display, edit, and delete records through a browser. Now we will reorganize the Python code so the routes can ask for those operations by name.

A route such as `post_create` knows that a form has arrived and that the browser should return to the list when the pet has been saved. The details of the insert belong in `database.py`. That gives us a useful boundary: the Flask application handles requests and responses, while the database module handles stored records.

The example is in `chapter-04-database-abstraction/code`. This chapter follows its `app.py`, `database.py`, and templates. Code excerpts use adjusted line breaks where needed.

Why an Abstraction Layer?

An **abstraction layer** gives us named operations and hides the details of carrying them out. In this application, the route can ask for a pet with one call:

```
pet = database.get_pet(id)
```

The route needs to know what to pass and what it will get back. Here, it supplies an identifier and receives a dictionary, or `None` when no pet has that identifier. The database module takes care of the cursor, the Structured Query Language (SQL) query, and the conversion of a row into a dictionary.

That makes a route easier to read. We can follow what the application is doing without working through a query every time it needs a record. The details are still available in `database.py` when we need them.

It also gives changes a place to go. We can revise the query inside `get_pet` while keeping the same arguments and return values. The calling route can stay as it is. Other routes and Python tests can use that same operation, so we have one implementation to maintain.

The Single Responsibility Principle

The **Single Responsibility Principle (SRP)** says that a module should have one reason to change. We group code by the responsibility it serves.

For our application, the division looks like this:

What changes?	Where it belongs
Request handling or page selection	<code>app.py</code>
How pets are stored or retrieved	<code>database.py</code>

Suppose we decide that saving a pet should send the browser to a different page. That change belongs in `app.py`, where we choose the response. Suppose we need a different query to load a pet. That belongs in `database.py`.

A responsibility can involve several functions. Reading, creating, updating, and deleting pets all belong to the database module because they serve its job of managing stored records. The functions give us useful operations within that responsibility.

Adding a field may still require changes to the database module and the forms. That feature changes both what we store and what the user enters. The principle helps us decide where each part belongs. Code that changes for the same reason stays together, and the interface connects the separate responsibilities.

Running the Example

From the topic directory, with Flask installed in your Python environment:

```
flask run
```

Open the list page:

```
http://127.0.0.1:5000/list
```

The application also accepts `/pets` for the list. Its redirects and template links use `/list`.

The database setup runs when the application starts. We can also run it separately:

```
python3 setup_database.py
```

That prepares `pets.db` in the working directory. To prepare a different file, supply its name:

```
python3 setup_database.py practice.db
```

The application itself still selects `pets.db`. The optional filename above belongs to the setup command. Make a working copy of the topic directory for exercises that change records.

What Moved into the Database Module?

The application imports the module and prepares its database:

```
import database

app = Flask(__name__)
database.setup_database("pets.db")
```

The routes now call a small set of functions:

Function	Work	Return value
<code>get_pets()</code>	Read the pet table	List of dictionaries
<code>get_pet(id)</code>	Read one pet	Dictionary, or None
<code>create_pet(data)</code>	Insert and commit	None
<code>update_pet(id, data)</code>	Update and commit	None
<code>delete_pet(id)</code>	Delete and commit	None

These functions form the interface between the application and its database code. The calling route supplies the values the operation needs. It receives a record or collection when it asks to read data; a write function completes its work before returning.

The SQL has a clear home now. Each function handles a database operation, while the route decides how that operation fits into a request and response.

Preparing the Table

The connection setup is:

```
def initialize(database_file):
    global connection
    connection = sqlite3.connect(database_file, check_same_thread=False)
    connection.row_factory = sqlite3.Row
    print("succeeded in making connection.")
```

`connection` is a module variable used by the database functions. `initialize` selects the file and configures how query results will be represented. We will use that row setting shortly.

`setup_database` calls `initialize` and creates the table when it is absent:

```
connection.execute(
    """create table if not exists pet (
        id integer primary key autoincrement,
        name text not null,
        type text not null,
        age integer,
        food text,
        owner text
    )""")
```

This version keeps `food` and adds `owner`. It uses the `name` type for the animal category, such as `dog` or `cat`; the introductory web example called that field `kind`. The forms and database functions use the field names shown here.

CREATE TABLE IF NOT EXISTS leaves an existing table in place. Setup then reads its column names and adds food or owner when one is missing:

```
columns = {
    row["name"]
    for row in connection.execute("pragma table_info(pet)")
}
for column in ("food", "owner"):
    if column not in columns:
        connection.execute("alter table pet add column " + column + " text")
connection.commit()
```

PRAGMA table_info(pet) asks SQLite for information about the table's columns. The set comprehension collects their names. The loop uses the two column names written in the program to construct the needed ALTER TABLE statements.

Running setup again preserves existing records. A newly added column has NULL for those records until a value is supplied. A fresh database starts with an empty pet table, ready for records entered through the form.

Records with Named Fields

The connection's row_factory setting makes each fetched result a sqlite3.Row. We can access a column using its name, and dict(row) converts it to an ordinary Python dictionary.

The list function uses that conversion:

```
def get_pets():
    cursor = connection.cursor()
    cursor.execute("""select * from pet""")
    pets = cursor.fetchall()
    pets = [dict(pet) for pet in pets]
    for pet in pets:
        print(pet)
    return pets
```

The final loop displays records in the server terminal. The returned list is what the route passes to its template.

A record has named fields like these illustrative values:

```
{
    "id": 3,
    "name": "Casey",
    "type": "dog",
    "age": 9,
    "food": "kibble",
    "owner": "Greg"
}
```

The template can write pet['id'] or pet['food'] directly. It no longer needs a tuple position to identify a field. That is especially convenient when we add a column such as owner.

The route for displaying the list is now short:

```
@app.route("/pets", methods=["GET"])
@app.route("/list", methods=["GET"])
def get_pets():
    return render_template("list.html", pets=database.get_pets())
```

There are two functions named `get_pets`: the route in `app.py` and the database function in `database.py`. The qualified name `database.get_pets()` tells Python which one to call.

Choosing the Display Order

The list template names the columns in the order it displays them:

```
{% for pet in pets %}
<tr>
    {% for key in ['id', 'name', 'type', 'age', 'food', 'owner'] %}
    <td>{{ pet[key] if pet[key] is not none else '' }}</td>
    {% endfor %}
    <td><a href="/delete/{{pet['id']}}">Delete</a></td>
    <td><a href="/update/{{pet['id']}}">Update</a></td>
</tr>
{% endfor %}
```

The outer loop walks through pets. The inner loop walks through field names. This keeps the cells aligned with the headings even when a column was added to an existing database after the table was first created.

A database NULL becomes Python None. The expression in the cell displays it as an empty string. The edit and delete links use the record's identifier, just as before.

Creating a Pet

The create form supplies name, age, type, food, and owner. These input names match the dictionary keys expected by `create_pet`.

The route obtains the submitted form data and delegates the save:

```
@app.route("/create", methods=["POST"])
def post_create():
    database.create_pet(dict(request.form))
    return redirect(url_for("get_pets"))
```

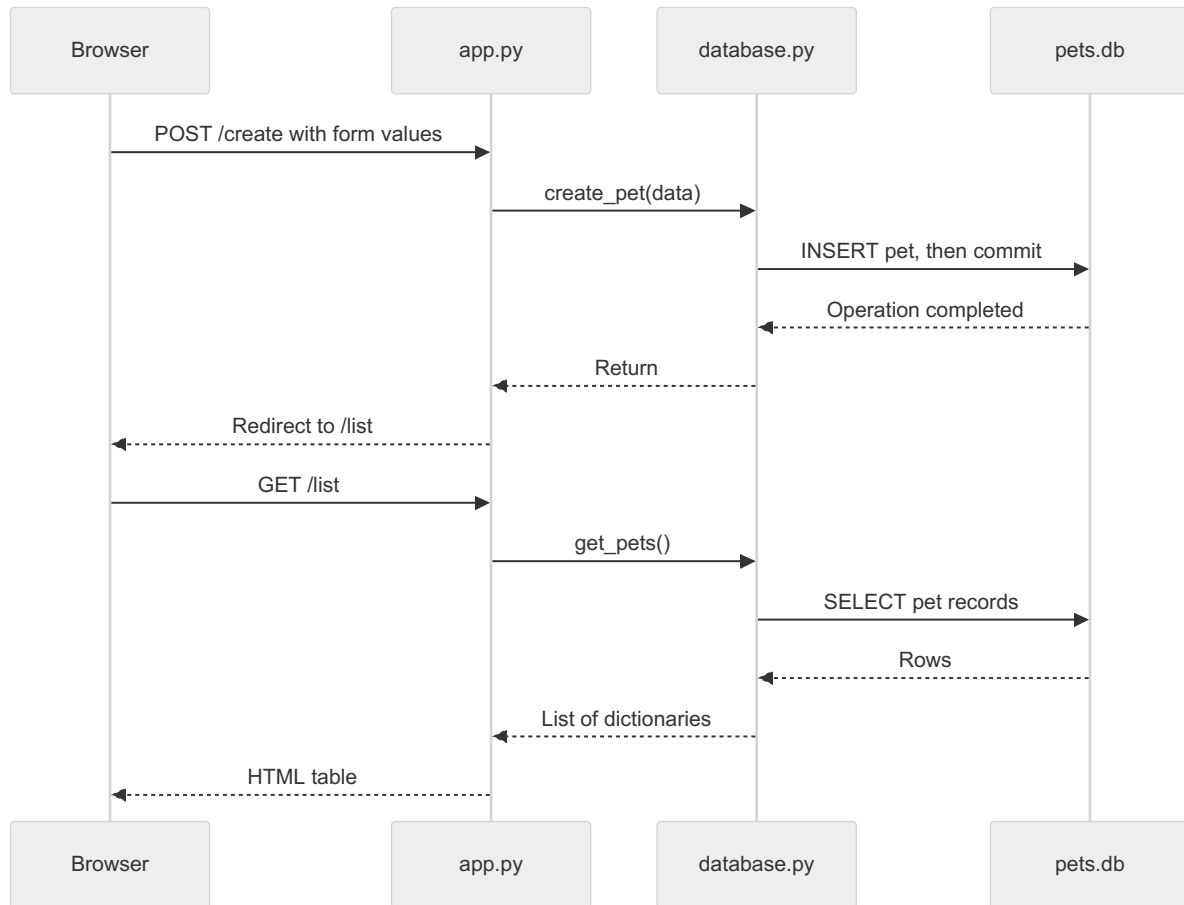
The function in `database.py` does the conversion and insertion:

```
def create_pet(data):
    try:
        data["age"] = int(data["age"])
    except:
        data["age"] = 0
    cursor = connection.cursor()
    cursor.execute(
        """insert into pet(name, age, type, food, owner)
           values (?, ?, ?, ?, ?)""",
        (data["name"], data["age"], data["type"],
         data["food"], data["owner"]),
    )
    connection.commit()
```

The submitted age arrives as text. The function converts it to an integer, using zero when conversion fails. The other fields are bound to the SQL statement in the order given by its column list. SQLite supplies the new identifier.

The function commits before returning. The route then redirects the browser to the list. Its `url_for("get_pets")` refers to the Flask endpoint in `app.py`; it does not call the database function with the same name.

Creating a Pet Through the Database Module



The extra lane makes the division of work visible. `app.py` handles the Hypertext Transfer Protocol (HTTP) requests and returns Hypertext Markup Language (HTML). `database.py` executes the database operations and returns Python values.

Loading One Pet

An edit begins with `get_pet(id)`:

```
def get_pet(id):
    id = int(id)
    cursor = connection.cursor()
    cursor.execute("""select * from pet where id = ?""", (id,))
    pet = cursor.fetchone()
    if pet is None:
        return None
    return dict(pet)
```

The identifier chooses one record. `fetchone()` returns a row when one exists, or `None` when the query found no matching record. The function preserves that distinction for its caller.

The route uses the result to choose a response:

```
@app.route("/update", methods=["GET"])
@app.route("/update/<int:id>", methods=["GET"])
def get_update(id=None):
    if id is None:
        return render_template(
            "error.html", error_message="No ID was provided."
        ), 400
    data = database.get_pet(id)
    if data is None:
        return render_template(
            "error.html", error_message="Data not found."
        ), 404
    return render_template("update.html", data=data)
```

Flask's `<int:id>` route converter supplies the identifier as an integer. The separate `/update` route handles a request with no identifier. The returned status `400` identifies a bad request; `404` identifies a record that was not found.

The database function reports whether it found a pet. The route decides which page and status to send. For a matching record, `update.html` fills the form using dictionary values such as `data['food']` and `data['owner']`.

Saving an Edit

The update route also checks that the selected pet exists. Then it calls:

```
database.update_pet(id, dict(request.form))
return redirect(url_for("get_pets"))
```

Inside `update_pet`, the age conversion follows the same pattern as creation. The database work is:

```
cursor = connection.cursor()
cursor.execute(
    """update pet set name=?, age=?, type=?, food=?, owner=?
       where id=?""",
    (data["name"], data["age"], data["type"],
     data["food"], data["owner"], id),
)
connection.commit()
```

The first five parameters supply the replacement values. The last selects the record. The commit completes the save before the route sends its redirect.

The persistence cycle is the same one we followed in the web application: select an action, load the state, edit it, and save it. What changed is where the work lives. We can now follow a call to `database.update_pet` whenever we need to see how an edit reaches the stored record.

Deleting a Pet

The delete route delegates its operation too:

```
@app.route("/delete/<int:id>", methods=["GET"])
def get_delete(id):
    database.delete_pet(id)
    return redirect(url_for("get_pets"))
```

The database function takes the identifier, executes a parameterized DELETE, and commits. It does not render a page or choose a redirect. Those decisions remain in the route.

This gives the write routes a consistent structure: obtain the input, call the database operation, and return the next response.

Testing Without the Browser

The module contains tests that call its functions directly. Run them from a working copy of the topic directory:

```
python3 database.py
```

The test setup selects `test_pets.db`, clears its pet rows, and inserts four sample records. The application database, `pets.db`, is separate from that test run.

`test_get_pets()` checks that the returned collection is a list of dictionaries with the expected keys.

`test_create_pet()` creates a record with food and owner values, reads it back, updates those values, and finally deletes it. Its closing check is:

```
delete_pet(pet["id"])
assert get_pet(pet["id"]) is None
```

These calls exercise the same functions used by the routes. We can verify the database behavior without filling in a browser form. A browser check then follows how requests and templates use that behavior.

Practice

Use a working copy of the topic directory.

1. Run the setup command twice. Confirm that the second run preserves existing records. Prepare a separate empty database with the optional filename and inspect its columns.
2. Start Flask and create a pet with both food and owner values. Find the record using the SQLite shell.
3. Trace the create request through `app.py` and `database.py`. Identify where form data becomes a dictionary, where the insert executes, and where the redirect is chosen.
4. Edit the pet's food and owner. Follow the calls that load the form and save the submitted values.

5. In a Python session, initialize a practice database and call `get_pet` with an identifier that is absent. Compare that return value with the page returned by an edit request for an absent record.
6. Add a `get_pets_by_owner(owner)` function using a parameterized query. Test it directly before adding a route that displays its results.
7. Add a unit test that creates a pet named O'Malley and verifies its food and owner fields after reading it back.
8. Change the order of fields in the list template and move the matching headings. Keep the database functions unchanged.

Exercise: Changing the Storage Behind the Interface

Our routes ask the database layer to load and save pets. In this exercise, we will replace the implementation behind those calls and run the same application. This lets us see what the abstraction layer buys us: the storage code can change while the routes and templates keep doing their jobs.

The alternative implementation stores each pet in a separate JavaScript Object Notation (JSON) file. The files make the change easy to inspect. We will focus on the interface the application uses and the behavior it expects.

Examine the Replacement

Work in a copy of `chapter-04-database-abstraction/code`. There are two new modules:

- `jsonbase.py` reads and writes dictionaries in a specified folder, with one numbered JSON file per record.
- `json_database_layer.py` implements the pet operations using that file store.

Compare `json_database_layer.py` with `database.py`. Both provide `setup_database`, `get_pets`, `get_pet`, `create_pet`, `update_pet`, and `delete_pet`. Check their parameters and return values. The routes expect a list of dictionaries from `get_pets`, and a dictionary or `None` from `get_pet`. The write functions complete the change before returning.

The implementation can differ because the caller works through these shared operations. Keeping the function names alone would not be enough. Their results must still mean what the application expects.

Switch the Import

Stop Flask. In your copy of `app.py`, replace:

```
import database
```

with:

```
import json_database_layer as database
```

The name `database` now refers to the replacement module. Calls such as `database.get_pet(id)` still work without edits to the routes.

Leave the existing setup call in place:

```
database.setup_database("pets.db")
```

The JSON adapter interprets that name as the folder `pets_json`. It leaves the SQLite file alone. It also accepts an explicit folder name, such as `practice_records`, but keep the original setup call for this comparison so the import is the only application change.

Start the application again:

```
flask run
```

Open <http://127.0.0.1:5000/list>. With a new `pets_json` folder, the list will be empty. Switching implementations selects a different store; it does not transfer the SQLite records.

Follow the Same Operations

1. Create a pet with food and owner values. Find its numbered file in `pets_json` and compare the saved fields with the values in the form.
2. Edit the pet. Confirm that the list shows the changes and that its file contains the new values.
3. Stop and restart Flask. Confirm that the pet still appears.
4. Delete the pet. Confirm that it disappears from the list and that its record file is gone.
5. Stop Flask, restore `import database`, and restart. Confirm that the original SQLite records are still available.

The JSON files include the integer `id` used in the application's edit and delete links. A small `.next-id` file tracks the next identifier, so deleting a record does not cause its identifier to be reused.

Check the Interface

Run the supplied tests from the code directory:

```
python3 -m unittest -v test_json_database.py
```

The tests use temporary folders. They run the same Flask operations with each implementation and start a separate Python process to check that saved records survive a restart. They also check missing records and the conversion of the age field.

Choose the update route and trace its calls with each import. Write a short explanation of which work stays in the route and which work changes inside the database layer. Then identify one change to the replacement's return values that would break the application, even though the function names stayed the same.

We changed the application's storage by changing one import. The abstraction layer made that possible by giving both implementations the same interface and expected behavior.

What Could Make `jsonbase` Faster?

Suppose the application grows from a few pets to 100,000 records. Look at `jsonbase.py` and suggest ways to improve its performance. Which operations would slow down as the folder grows? Where does the code scan filenames or read more data than an operation needs?

Choose one improvement to investigate. Explain which operation it would help, what extra work or storage it might require, and how you would measure whether it helped. Consider what happens when records change as well as when they are read.

We will return to these questions in future classes as we study database performance. For now, propose ideas and the measurements you would use to compare them.

Questions for Thought and Study

1. What does a route need to know to call `create_pet`?
2. Which decisions belong to `app.py`, and which belong to `database.py`?
3. What does `connection.row_factory = sqlite3.Row` change?
4. Why does the module convert fetched rows to dictionaries?
5. How does `get_pet` represent a missing record, and how does the route use that result?
6. What makes the list template's column order independent of the table's physical column order?
7. Why does the write function commit before returning to the route?
8. How can the same database functions serve both a web application and a Python test?

Further Reading and Practice

- **Python SQLite interface:** Connections, row objects, parameters, and fetching results.
<https://docs.python.org/3/library/sqlite3.html>
- **SQLite table information:** The `table_info` command used during setup.
https://www.sqlite.org/pragma.html#pragma_table_info
- **SQLite ALTER TABLE:** Adding columns to an existing table.
https://www.sqlite.org/lang_altertable.html
- **Flask quickstart:** Routes, converters, templates, and redirects.
<https://flask.palletsprojects.com/en/stable/quickstart/>
- **Abstraction:** Background on hiding implementation details behind an interface.
[https://en.wikipedia.org/wiki/Abstraction_\(computer_science\)](https://en.wikipedia.org/wiki/Abstraction_(computer_science))